🔒

Distributed Lock

分布式系統中的資料競爭問題

在分布式系統中，你是否遇過資料「莫名其妙」被覆蓋？或是同一筆資料的更新結果衝突不一致？這類問題常常源自兩個以上的服務實例同時對同一份資料進行讀寫。

這種情況下，如果沒有協調機制，資料的一致性、正確性就會受到威脅。分布式鎖就是一種常用的解決方法，用來確保在同一時間只有一個實例能夠對共享資源進行操作。

以下是在系統設計面試中常見的分散式鎖使用場景：

1. 電子商務結帳系統（E-Commerce Checkout System）

使用分散式鎖，在使用者結帳期間（例如 10 分鐘），暫時保留一項高需求商品，例如限量球鞋，以確保當某位使用者正在完成付款流程時，該商品不會被其他人同時購買。

2. 叫車媒合系統（Ride-Sharing Matchmaking）

分散式鎖可用來管理司機與乘客的配對。當乘客發出叫車請求時，系統可以鎖定附近的一位司機，避免該司機同時被配對給多位乘客。這個鎖會持續到司機確認或拒絕該趟行程，或直到經過一定時間為止。

3. 分散式排程任務（Distributed Cron Jobs）

對於在多台伺服器上執行排程任務（cron jobs）的系統，分散式鎖可以確保某個任務同一時間只會被一台伺服器執行。例如，在資料分析平台中，每日任務會彙總使用者資料產生報表。分散式鎖可以防止該任務在多台伺服器上重複執行，以節省運算資源。

4. 線上拍賣競標系統（Online Auction Bidding System）

在拍賣的最後階段，分散式鎖可用來確保當最後幾秒內有新的出價時，系統會短暫鎖定該商品以處理該出價並更新目前最高出價，避免其他使用者同時對同一商品出價。

為什麼需要分布式鎖？

bulldmoat.org

當系統擴展到多台機器或多個微服務之後，共享資源（像資料庫、檔案、API 等）同時被多個實例訪問就變得很普遍。如果沒有機制來協調這些訪問，就會發生例如：

- 多個 Writer 同時修改同一行資料，其中一個覆蓋掉另一個的變更
- 消費者從消息系統收到重複訊息時，可能會重複處理
- 需要避免多台節點同時執行同一個定時任務

在這些場景中，分布式鎖提供了一種簡單直觀的方式：**在同一時間只有一個節點可以操作該資源**。

這類似於一種「簡易版的領導者選舉」——如果你只需要確保一次只有一個實例運行某個任務，那麼分布式鎖往往比完整的領導者選舉機制更輕量。

分布式鎖的基本運作流程

分布式鎖的核心邏輯其實相當簡單，可以分成以下步驟：

1. **請求鎖**

   節點判斷自己需要對某個資源進行獨佔式操作，向支援 TTL 或「臨時鎖」機制的鎖管理者（例如 Redis、ZooKeeper、資料庫）發出請求。

2. **取得鎖的嘗試**

   - 當當前沒有鎖存在時，節點成功建立鎖並擁有該資源
   - 如果已有其它節點持有鎖，則根據策略選擇「等待」、「失敗立即返回」或「輪詢重試」

3. **執行臨界區任務**

   一旦取得鎖，節點進行必須獨佔的操作，例如更新資料庫行、寫檔、調用限制 API 等。

4. **正常釋放鎖**

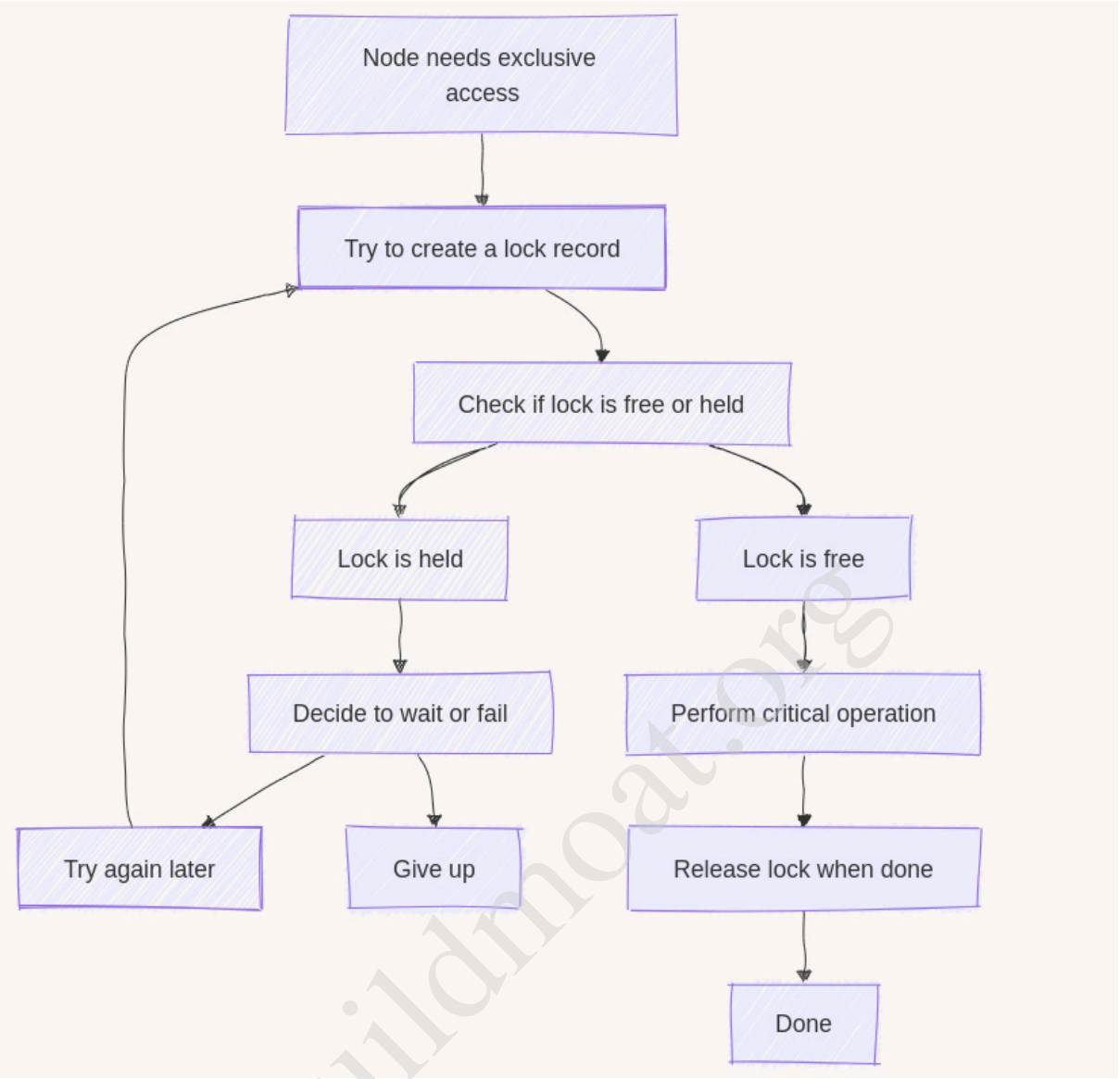
   任務完成後刪除鎖（例如刪掉 Redis 的 key），讓其他候選者能拿到鎖。

5. **當機或 TTL 過期機制**

   如果持有鎖的節點當機或 Session 斷開，TTL 過期或臨時節點機制會自動釋放鎖，避免「殭屍鎖」永久阻塞系統。

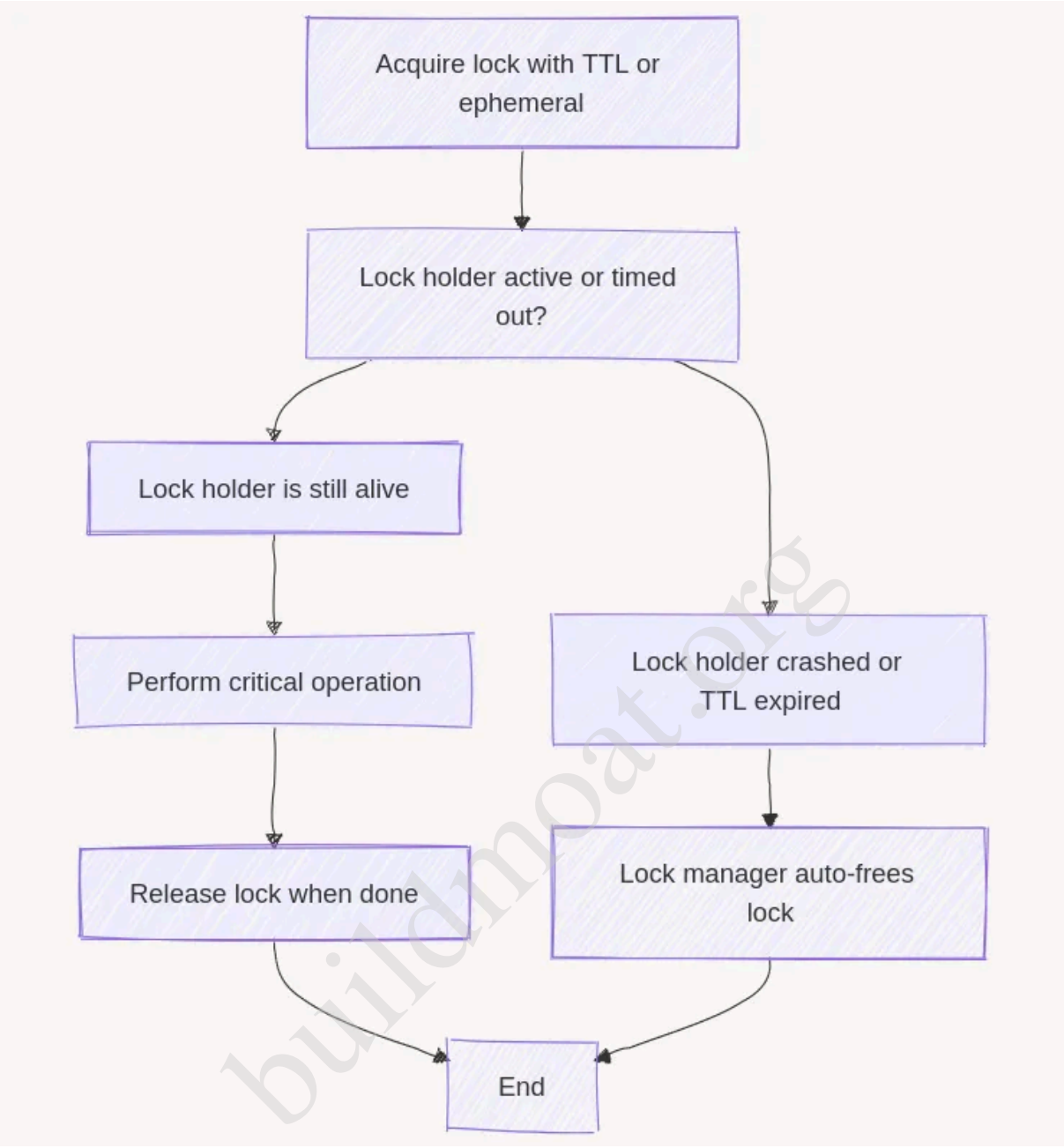
基本的操作流程如下：

buildmoat.org



buildmoat.org

在有 TTL 處理的情況下如下：



buildmoat.org

常見的分布式鎖實作工具與策略

分布式鎖本質上依賴某種一致性機制來協調鎖的狀態。下面是幾種常見的選擇：

1. Redis 鎖（基於 TTL）

Redis 是一種常被用作分布式鎖的快取系統，因為它提供了原子操作與 TTL（過期時間）功能。主要方式如下：

- 使用 `SET lockKey value NX EX seconds`：NX 表示只有鍵不存在時才設置，EX 表示設定 TTL

- 如果 `SET` 返回成功 (OK)，表示鎖成功被取得
- 使用 TTL 防止死鎖：當鎖超時後 Redis 自動移除 key ，避免節點宕機後鎖無法釋放
- 持有鎖的節點完成任務後，執行 `DEL lockKey` 釋放鎖
- 在 TTL 內如果需要更多時間，可以延長 TTL（但必須確保仍為鎖的擁有者）

這種方式簡單、容易整合到現有系統，但在網路分裂或 Redis Cluster 異常時可能出現安全性問題。為了加強保證，可引用 Redlock 等分布式 Redis 鎖算法來協調多個 Redis 節點。

2. ZooKeeper / etcd（使用臨時節點）

ZooKeeper 和 etcd 都是設計為強一致性的分散式 key-value 系統，它們支援**臨時節點（Ephemeral Nodes）**：

- 建立一個臨時節點表示取得鎖：如果這個節點存在，就表示有人在持有鎖
- 當客戶端離線、Session 失效或當機時，該臨時節點會被清除
- 其他候選者可以監聽該臨時節點的變化，當它消失時嘗試重新取得鎖

因為資料在多個節點之間 Replicate，這種方式在網路分割或節點失效時能提供更強的一致性保證。

3. 資料庫鎖（Advisory Locks / Row Locks）

如果系統已經有一個單一資料庫（例如 Postgres、MySQL），也可以利用現有資料庫的鎖機制來協調：

- **行級鎖（Row-Level Locks）**

  使用 SQL 的 `SELECT … FOR UPDATE` 在交易內鎖住某一行直到 Commit/rollback

- **Advisory Locks**

  資料庫提供的命名鎖，例如 Postgres 的 `pg_advisory_lock()` 或 MySQL 的 `GET_LOCK()`，可以透過自訂識別符號來管理鎖

這種方式無需引入額外基礎建設(e.g. Redis or Zookeeper)，但只適合單一 DB 範圍的場景，跨資料庫或多 Region 拓展性差。

4. Kubernetes 單實例部署（避免鎖）

在某些情況下，你甚至不需要鎖機制本身，只要確保某根本沒有多個實例在競爭：

- 在 Kubernetes 設定 Deployment/StatefulSet 的 replicas 為 1

buildmeat.org

因此只會有一個 Pod 實例在運行，不會出現競爭

這其實不是一個真正意義上的分布式鎖，而是藉由**運行環境強制單一實例**來避開鎖的需求，適合一些不追求高可用性但需要簡單方案的場景。

分布式鎖的常見陷阱與最佳實踐

1. 死鎖與鎖順序問題（Deadlocks）

如果系統需要一次取得多個鎖，會存在死鎖的風險（A 拿了鎖 1 想拿鎖 2，而 B 先拿了鎖 2 想拿鎖 1），因此：

- 應該在全系統統一的順序下取得鎖
- 使用適當設計的交易邊界

2. 鎖競爭（Contention）

過多的服務同時競爭相同鎖會導致系統執行變慢，因此：

- 只在必要的最小臨界區使用鎖
- 根據情況考慮分片或細粒度的狀態鎖

3 宕機與鎖過期策略

- 使用 TTL 或臨時節點 (ephemeral node) 避免「殭屍鎖」
- 設計合理的 Timeout 和釋放邏輯

4. 鎖管理者的單點故障

若鎖管理者本身是單一節點，那它就成為了單點故障，例如：

- 單一 Redis 實例失效就使鎖失效
- 最佳做法是用 Redis Cluster、Sentinel 或多節點 ZooKeeper/etcd

5. 時鐘偏差 (Clock Skew) 與網路分區 (Network Partitions)

即使 Redlock 等演算法試圖處理部分失敗，分布式鎖在極端網路分區情況下也可能出現不一致性（CAP 定理）。

總結與設計考量

分布式鎖是協調跨系統並發（concurrency）的基礎工具之一。

buildmoat.org

不論你選擇的是：

- 使用 Redis 搭配 TTL 的鎖機制
- 使用 ZooKeeper 的 ephemeral nodes
- 使用資料庫鎖（row-level 或 advisory locks）
- 或甚至在 Kubernetes 中只運行單一副本

其核心原則始終相同：**在同一時間，只允許一個實體對某個資源進行操作。**

在這個模型中，「如何處理失敗」是關鍵。

如果持有鎖的節點崩潰，而沒有適當機制釋放鎖，整個系統可能會卡死。因此，ephemeral（會隨 session 消失）或 time-based（TTL 到期）機制成為標準設計，而不是選配。

不同方案在複雜度、效能、容錯能力上各有取捨，沒有一種方式是普世最佳解。

1. 先問自己：真的需要分布式鎖嗎？

盡可能避免使用分布式鎖。

很多時候，鎖本身是一個「設計警訊」，提醒你應該重新審視：

- 為什麼會走到需要鎖的地步？
- 是否可以透過架構調整（例如單寫入者模型、事件冪等設計）來避免？

鎖應該是經過審慎思考後的選擇，而不是直覺反應。

2. 小心使用鎖：過度上鎖會壓垮併發能力 (concurrency)

鎖如果用太多、範圍太大，會嚴重降低系統的併發處理能力。

但同時，在某些場景中鎖又是不可或缺的，例如：

- 防止資料毀損
- 防止重複處理（double processing）
- 保證交易一致性

因此關鍵不是「要不要鎖」，而是：

- 鎖的粒度是否合理？
- 臨界區是否縮到最小？
- 是否真的只保護必要部分？

3. 選擇合適的儲存系統來實現鎖

buildmoat.org

不同工具適用於不同情境：

- **Redis**

  如果你本來就有在使用 Redis，它通常是最簡單、整合成本最低的選擇。

- **ZooKeeper**

  適合需要更進階協調能力的場景，例如 cluster-level coordination。

- **資料庫鎖**

  在單一區域（single-region）或單體架構中，往往已經足夠，而且不需要引入額外基礎設施。

選擇工具時，不只是看功能，而是看你的架構規模與失敗模型。

4. 必須設計失敗處理機制

務必實作 TTL 或 ephemeral 機制，以避免「殭屍鎖」（zombie locks）或卡住的鎖。

持有鎖的節點崩潰，是常態，而不是例外。

你的設計必須假設失敗會發生。

5. 如果完全不需要並發？

在 Kubernetes 中，如果你把 Deployment 或 StatefulSet 設為：

```
replicas: 1
```

那麼並發問題就消失了。因為根本沒有多實例競爭。

但要清楚：

- 這不是分布式鎖
- 這只是營運層級的限制
- 你放棄了水平擴展能力

這種方式適合簡單場景，但不適合需要高可用與擴展性的系統。

6. 理解權衡，而不是迷信模式

如果場景很單純，一個基於鎖的「單一訂閱者」模型，可能比完整的 leader election 更簡單。

但如果你在做進階的叢集協調，真正的 leader election 或成熟的分布式協調方案，可能會更健壯。

buildmoat.org

不要為了技術純度而選擇過度複雜的方案。

最後的思考

分布式系統永遠伴隨複雜性。

你無法消除它，但可以馴服它。

一個設計良好的分布式鎖（或策略性地選擇單實例運行）能有效控制並發混亂：

- 保持資料一致性
- 防止競爭條件
- 穩定整體架構

鎖不是萬靈丹，但在正確的位置，它是讓系統維持秩序的重要工具。

buildmoat.org